05 — The Query Planner

> *"The planner is PostgreSQL's brain — understanding it lets you work with it, not against it."*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Describe the phases of query planning (parse, rewrite, plan, execute)
- Explain how statistics in pg_statistic are used for cost estimation
- Interpret cost parameters and their effect on plan selection
- Identify why a planner makes a wrong choice and how to fix it
- Use planner configuration parameters appropriately
- Understand generic vs custom plans for prepared statements

Planner Architecture Overview

PostgreSQL query processing has four phases:

1. **Parse**: Convert SQL text to a parse tree (syntactic checking)
2. **Analyze/Rewrite**: Semantic analysis, view expansion, rule application
3. **Plan**: Generate and evaluate alternative plans, choose the cheapest
4. **Execute**: Execute the chosen plan

The **planner** operates in phase 3. It:

1. Generates candidate plans (all possible scan + join combinations)
2. Estimates costs for each plan using statistics

3. Returns the plan with the lowest estimated cost

ASCII Diagram: Query Planning Pipeline

```
SQL Text:
"SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id WHERE o.status =
'pending'"

Phase 1: PARSER
┌──────────────────────────────────────────────────────┐
│  Parse Tree:                                         │
│  SELECT                                              │
│    FROM: orders (alias o), customers (alias c)       │
│    WHERE: o.status = 'pending'                       │
│    JOIN: o.customer_id = c.id                        │
└──────────────────────────────────────────────────────┘

Phase 2: ANALYZER / REWRITER
┌──────────────────────────────────────────────────────┐
│  Query Tree:                                         │
│  - Resolve table OIDs (orders → pg_class.oid=12345)  │
│  - Resolve column types (status → text, customer_id → int)│
│  - Expand views if any                               │
│  - Apply rewrite rules                               │
└──────────────────────────────────────────────────────┘

Phase 3: PLANNER (cost-based optimizer)
┌──────────────────────────────────────────────────────┐
│  1. Gather statistics from pg_statistic              │
│     orders: 1,000,000 rows, 100 pages                │
│     customers: 50,000 rows, 25 pages                 │
│     status: 3 distinct values, 'pending' = 5% of rows │
│                                                      │
│  2. Generate alternative sub-plans for each table:   │
│     orders:    Seq Scan, Index Scan (status), ...    │
│     customers: Seq Scan, Index Scan (id/PK), ...     │
│                                                      │
│  3. Consider join algorithms:                        │
│     Plan A: Nested Loop (orders → idx scan customers) │
│     Plan B: Hash Join (hash customers, probe with orders)│
│     Plan C: Merge Join (sort both, merge)            │
│                                                      │
│  4. Calculate costs:                                 │
│     Plan A: 0.56 + 50000 × (0.43 + 1.12) = 77,543    │
│     Plan B: 1234 + 50000 × 0.01 = 1,734  ← CHEAPEST! │
│     Plan C: 12345 + ... = 23,456                     │
│                                                      │
│  5. Choose Plan B (Hash Join)                        │
└──────────────────────────────────────────────────────┘

Phase 4: EXECUTOR
```

```
┌──────────────────────────────────────────────────────┐
│  Execute Hash Join plan                              │
│  1. Build hash table from customers (50K rows)       │
│  2. Scan orders, filter status='pending' (50K rows)  │
│  3. Probe hash table for each pending order          │
│  4. Return results                                   │
└──────────────────────────────────────────────────────┘
```

Statistics: pg_statistic and pg_stats

PostgreSQL maintains statistics about table data in `pg_statistic` (raw) and `pg_stats` (human-readable view).

pg_stats Columns

```sql
SELECT *
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'customer_id';
```

Column	Description	Use in Planning
`null_frac`	Fraction of NULLs	Adjust row estimates for IS NULL
`avg_width`	Average column width in bytes	Memory estimation
`n_distinct`	Distinct values count (negative = fraction of total)	Estimate selectivity for =
`most_common_vals`	Array of most frequent values	Accurate estimates for common values
`most_common_freqs`	Frequencies for most_common_vals	
`histogram_bounds`	Bucket boundaries for range estimates	Range predicate estimation
`correlation`	Physical vs logical order correlation	Index scan cost, BRIN decisions
`most_common_elems`	For arrays: common elements	
`elem_count_histogram`	For arrays: element count distribution	

Inspecting Statistics

```sql
-- Check statistics for a specific column
SELECT
    attname,
    null_frac,
    avg_width,
    n_distinct,
    most_common_vals,
    most_common_freqs,
```

```
    histogram_bounds,
    correlation
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'status';
```

Example output:

```
attname: status
null_frac: 0
avg_width: 10
n_distinct: 3
most_common_vals: {completed,pending,processing}
most_common_freqs: {0.85,0.10,0.05}
histogram_bounds: NULL (no histogram for low-cardinality columns)
correlation: 0.002 (random, status doesn't correlate with physical order)
```

The planner uses `most_common_freqs` to estimate:

- `WHERE status = 'completed'` → selectivity = 0.85 (85% of rows)
- `WHERE status = 'pending'` → selectivity = 0.10 (10% of rows)

Selectivity Estimation

For Equality Predicates

```
If value is in most_common_vals:
    selectivity = most_common_freqs[index]
Else:
    selectivity = (1 - sum(most_common_freqs)) / (n_distinct -
length(most_common_vals))
```

For Range Predicates

```
If predicate falls within histogram:
    selectivity = (bucket_fraction × range_width) / total_range_width
```

For Multiple Predicates (Independence Assumption)

```
P(A AND B) = P(A) × P(B)  ← assumes column independence!
```

This is wrong when columns are correlated! Extended statistics fix this.

```
-- Example: 10% of orders are 'pending', 30% are in 'US'
-- Planner estimates: WHERE status='pending' AND region='US'
-- = 0.10 × 0.30 = 0.03 (3%)
-- Actual: maybe 15% because pending orders skew toward US customers

-- Fix: extended statistics for correlated columns
```

```
CREATE STATISTICS stats_status_region ON status, region FROM orders;
ANALYZE orders;
```

Statistics Target

The `statistics_target` controls how much data ANALYZE collects.

```
-- Default: 100 (most_common_vals array has up to 100 entries, histogram has 100
buckets)
SHOW default_statistics_target;

-- Increase for highly skewed or high-cardinality columns:
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
ANALYZE orders;
-- Now customer_id has up to 500 most-common values and 500 histogram buckets
-- Better estimates for queries on customer_id
```

Cost Parameters

Cost parameters control how the planner weights different operations.

Standard Cost Parameters

```
-- I/O costs
SHOW seq_page_cost;            -- default 1.0 (baseline)
SHOW random_page_cost;         -- default 4.0 (HDD); set 1.1 for SSD

-- CPU costs
SHOW cpu_tuple_cost;           -- default 0.01
SHOW cpu_index_tuple_cost;     -- default 0.005
SHOW cpu_operator_cost;        -- default 0.0025

-- Parallel query costs
SHOW parallel_setup_cost;      -- default 1000
SHOW parallel_tuple_cost;      -- default 0.1

-- Memory size hints
SHOW effective_cache_size;     -- default 4GB (should be 75% of total RAM)
SHOW work_mem;                 -- default 4MB
```

Effect on Plan Choice

```
random_page_cost = 4.0 (HDD default):
  → Index scan expensive relative to seq scan
  → Planner prefers seq scan for moderately selective queries
  → Crossover at ~1-5% selectivity

random_page_cost = 1.1 (SSD):
  → Index scan nearly as cheap as seq scan per page
```

```
  → Planner prefers index scan for much larger fractions
  → Crossover at ~20-50% selectivity

Recommendation: SET random_page_cost = 1.1 on ANY SSD storage!
```

Configuring for Your Hardware

```
-- SSD storage
ALTER SYSTEM SET random_page_cost = 1.1;
ALTER SYSTEM SET effective_cache_size = '24GB';  -- 75% of 32 GB RAM

-- HDD/SATA spinning disk
ALTER SYSTEM SET random_page_cost = 4.0;
ALTER SYSTEM SET effective_cache_size = '12GB';  -- 75% of 16 GB RAM

-- Apply:
SELECT pg_reload_conf();
```

Join Ordering (Dynamic Programming)

For queries with N tables, PostgreSQL tries all join orderings up to `join_collapse_limit`.

Dynamic Programming Phase

1. For each individual table: generate optimal single-table scan plans
2. For each pair: try all join combinations (NL, Hash, Merge) of optimal single-table plans
3. For each triple: extend the best pair plans with the third table
4. Continue until all N tables are joined

Complexity

- For N tables: 2^N possible join orderings
- PostgreSQL uses DP to avoid redundant work: O(3^N) with pruning
- Default `join_collapse_limit = 8` → at most 8 tables use full DP

For Many-Table Queries

```
-- Queries with > 8 tables use GEQO (genetic algorithm)
-- For complex queries that GEQO handles poorly, try:
SET join_collapse_limit = 20;  -- expand full DP search
SET geqo_threshold = 20;       -- delay switch to GEQO
```

Planner Configuration Parameters

Key Parameters to Know

```
-- Join search
SHOW join_collapse_limit;      -- default 8 (full DP up to 8 tables)
SHOW from_collapse_limit;      -- default 8 (subqueries can be folded into parent)
```

```
-- Aggregation
SHOW enable_hashagg;           -- use hash aggregate
SHOW enable_sort;              -- use explicit sort

-- Scans
SHOW enable_seqscan;
SHOW enable_indexscan;
SHOW enable_indexonlyscan;
SHOW enable_bitmapscan;
SHOW enable_tidscan;

-- Joins
SHOW enable_nestloop;
SHOW enable_hashjoin;
SHOW enable_mergejoin;

-- Parallel
SHOW enable_parallel_query;
SHOW max_parallel_workers_per_gather;
SHOW min_parallel_table_scan_size;
SHOW min_parallel_index_scan_size;
```

When the Planner Makes Wrong Choices

Root Causes of Bad Plans

1. **Stale statistics**: ANALYZE not run after data changes
   - Symptom: `actual rows` >> `estimated rows` in EXPLAIN ANALYZE
   - Fix: `ANALYZE table_name;` or tune autovacuum
2. **Wrong hardware parameters**: `random_page_cost` set for HDD on SSD server
   - Symptom: Index scans avoided for selective queries on SSD
   - Fix: `SET random_page_cost = 1.1;`
3. **Incorrect `effective_cache_size`**: set too low
   - Symptom: Index scans appear too expensive
   - Fix: `SET effective_cache_size = '24GB';` (75% of RAM)
4. **Data correlation**: planner assumes column independence
   - Symptom: Nested loop with wrong outer/inner designation
   - Fix: `CREATE STATISTICS ON col1, col2 FROM table; ANALYZE;`
5. **Function on indexed column**: index not used
   - Symptom: Seq Scan despite index on column
   - Fix: Expression index or rewrite query without function
6. **statistics_target too low**: column has many distinct values, histogram is coarse
   - Symptom: Range predicate estimates off by 10-100×

- Fix: `ALTER TABLE t ALTER COLUMN c SET STATISTICS 500; ANALYZE t;`

Forcing Specific Plans

For debugging and testing purposes only:

```
-- Force/disable specific access methods
SET enable_seqscan = off;          -- force index usage
SET enable_indexscan = off;        -- prefer bitmap scan
SET enable_indexonlyscan = off;    -- disable index-only scan
SET enable_bitmapscan = off;       -- force regular index scan

-- Force/disable specific join algorithms
SET enable_nestloop = off;
SET enable_hashjoin = off;
SET enable_mergejoin = off;

-- Force join order
SET join_collapse_limit = 1;    -- use exact SQL JOIN order
SET from_collapse_limit = 1;    -- don't reorder FROM list

-- After testing, restore:
RESET ALL;
-- or reset individually:
SET enable_seqscan = on;
```

Never leave these disabled in production. Use `SET LOCAL` or `SET` in a transaction to scope changes to a specific query.

---

pg_hint_plan Extension

`pg_hint_plan` allows SQL hints to guide the planner — similar to Oracle hints.

```
CREATE EXTENSION IF NOT EXISTS pg_hint_plan;

-- Force specific join algorithm:
/*+ HashJoin(a b) */
SELECT * FROM orders a JOIN customers b ON a.customer_id = b.id;

-- Force specific scan type:
/*+ IndexScan(orders idx_orders_status) */
SELECT * FROM orders WHERE status = 'pending';

-- Force join order:
/*+ Leading(orders customers) */
SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id;

-- Combine hints:
/*+ HashJoin(o c) IndexScan(o idx_orders_status) */
```

```
SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'pending';
```

Available hints:

- `SeqScan(table)`, `IndexScan(table index)`, `IndexOnlyScan(table index)`, `BitmapScan(table)`
- `NestLoop(t1 t2)`, `HashJoin(t1 t2)`, `MergeJoin(t1 t2)`
- `Leading(t1 t2 t3)` — specifies join order
- `Rows(table #N)` — override row count estimate

EXPLAIN (GENERIC_PLAN) — PostgreSQL 16+

PostgreSQL 16 added `EXPLAIN (GENERIC_PLAN)` to show the plan that would be used for a prepared statement (generic plan, without parameter-specific optimization).

```
-- PostgreSQL 16+
PREPARE myquery AS SELECT * FROM orders WHERE customer_id = $1;
EXPLAIN (ANALYZE, GENERIC_PLAN) EXECUTE myquery(42);

-- Shows the generic plan (used after 5 executions with varied parameters)
-- vs the custom plan (optimized for specific parameter value)
```

Generic vs Custom Plans

- First 5 executions: PostgreSQL uses **custom plans** (optimized for specific parameter)
- After 5 executions: PostgreSQL compares generic vs custom plan costs
- If generic is cheaper (or within threshold): switches to generic
- `plan_cache_mode = force_generic_plan` or `force_custom_plan` to override

Common Mistakes

1. Blaming the planner without checking statistics

Before overriding the planner, run:

```
ANALYZE table_name;
EXPLAIN (ANALYZE, BUFFERS) query;
```

90% of "planner bugs" are stale statistics.

2. Setting `enable_seqscan = off` permanently

```
-- WRONG in production config (postgresql.conf or postgresql.auto.conf):
enable_seqscan = off
-- This breaks all full-table queries, forces bad plans
```

3. Setting `random_page_cost = 1` instead of `1.1` for SSD

While `1.0` might seem right for "same as sequential", SSD random reads are slightly more expensive due to internal overhead. `1.1` is the standard recommendation.

4. Not using extended statistics for correlated columns

```
-- Without extended statistics:
WHERE city = 'New York' AND state = 'NY'
-- Planner estimates P(city) × P(state) = 0.01 × 0.20 = 0.002 (wrong!)
-- Actual: ~1% of rows (similar but for wrong reason)

-- Fix:
CREATE STATISTICS stats_city_state ON city, state FROM addresses;
ANALYZE addresses;
-- Now planner estimates P(city='New York' AND state='NY') correctly
```

5. Ignoring `effective_cache_size` configuration

Leaving at the 4 GB default on a 128 GB server causes the planner to significantly underestimate the benefit of index scans (assumes most data must be read from disk).

Best Practices

1. **Configure `random_page_cost` and `effective_cache_size` correctly** — these two settings have the largest single impact on plan quality.
2. **Run ANALYZE after bulk data operations** — VACUUM ANALYZE does both.
3. **Increase `statistics_target` for highly-skewed or high-cardinality columns**.
4. **Use extended statistics** for tables with correlated columns that are often queried together.
5. **Monitor for plan regressions** after data growth — a plan that was correct at 100K rows may be wrong at 100M rows.
6. **Use `auto_explain`** with a long execution threshold to capture slow queries with their plans automatically.
7. **Never hardcode planner hints in application code** without a periodic review — data distributions change.

Performance Considerations

Planning Time

Complex queries can take significant time to plan:

```
EXPLAIN (ANALYZE, SUMMARY) complex_query;
-- Planning Time: 450 ms  ← if this is large, reduce query complexity
-- Execution Time: 100 ms
```

For frequently-run complex queries, use prepared statements to amortize planning cost:

```
PREPARE daily_report AS SELECT ... FROM ... WHERE date = $1;
-- Plan computed once, reused for subsequent EXECUTE calls
```

Prepared Statement Plan Caching

Prepared statements may use a "generic plan" that's not optimized for specific parameter values after the first few executions. This can cause performance regressions for highly skewed data distributions:

```
-- Force custom plan for each execution (re-plans every call):
SET plan_cache_mode = force_custom_plan;

-- Force generic plan (plan once, reuse):
SET plan_cache_mode = force_generic_plan;

-- Let PostgreSQL decide (default):
SET plan_cache_mode = auto;
```

Interview Questions & Answers

Q1. What is the PostgreSQL query planner and how does it work?

A: The PostgreSQL query planner is a cost-based optimizer that generates the optimal execution plan for a SQL query. It works in these steps: (1) Enumerates all possible access paths for each table (sequential scan, available index scans). (2) Enumerates all possible join orderings for multi-table queries using dynamic programming. (3) Estimates the cost of each plan using statistics from pg_statistic and cost parameters (seq_page_cost, random_page_cost, etc.). (4) Selects the plan with the lowest estimated total cost. The quality of the plan depends entirely on the accuracy of the statistics.

Q2. What is in pg_stats and how does the planner use it?

A: `pg_stats` (the user-friendly view of `pg_statistic`) contains: `null_frac` (fraction of NULLs), `avg_width` (average row width), `n_distinct` (distinct value count), `most_common_vals/freqs` (top values and their frequencies), `histogram_bounds` (range distribution buckets), and `correlation` (physical ordering vs logical ordering).

The planner uses these to estimate selectivity: for equality predicates on common values it uses `most_common_freqs`; for range predicates it uses `histogram_bounds`; for multiple predicates it multiplies individual selectivities (independence assumption). Wrong or outdated statistics are the primary cause of bad execution plans.

Q3. Why does the planner sometimes make wrong choices for correlated columns?

A: The planner assumes column independence when estimating selectivity for multiple predicates. `P(A AND B) = P(A) × P(B)`. For correlated columns (e.g., city and zip_code where city='NYC' implies zip codes are in the 10000s range), this underestimates the selectivity. The planner may think a query returns far fewer rows than it does, potentially choosing a Nested Loop when Hash Join would be better. The fix is to create extended statistics: `CREATE STATISTICS ON city, zip_code FROM addresses; ANALYZE addresses;` which teaches the planner about the joint distribution.

Q4. What is the difference between `join_collapse_limit` and `from_collapse_limit`?

A: `join_collapse_limit` controls how many tables can be involved in a JOIN before the planner stops trying all join orderings and uses a heuristic. Above the limit, it falls back to the order specified in the SQL. `from_collapse_limit` controls how deeply subqueries in the FROM clause can be "pulled up" (folded into the parent query) before being treated as a separate sub-plan. Both default to 8. Increasing them allows more aggressive optimization at the cost of longer planning time.

Exercises with Solutions

Exercise 1

After migrating 50M new rows into `orders`, all queries on this table became 10× slower. What would you check first?

Solution:

```
-- Step 1: Check when table was last analyzed
SELECT last_analyze, last_autoanalyze, n_live_tup, n_dead_tup
FROM pg_stat_user_tables WHERE relname = 'orders';
-- If last_analyze was before the migration: statistics are stale!

-- Step 2: Update statistics
ANALYZE orders;

-- Step 3: Verify estimates are now correct
EXPLAIN (ANALYZE, BUFFERS) slow_query;
-- Check actual rows vs estimated rows

-- Step 4: If still wrong, increase statistics target
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
ANALYZE orders;
```

Production Scenarios

Scenario 1: Sudden Plan Regression After Data Migration

A critical query that ran in 200ms started taking 45 seconds after migrating data from a legacy system. The data doubled from 50M to 100M rows but ANALYZE was not run.

```
-- Investigation:
EXPLAIN (ANALYZE, BUFFERS) slow_query;
-- Found: rows=500 (estimate) vs actual=50000 (100x underestimate!)
-- Planner chose Nested Loop (appropriate for 500 rows) → catastrophic for 50K

-- Fix:
ANALYZE orders;
EXPLAIN (ANALYZE, BUFFERS) slow_query;
-- Now: rows=50000 (estimate) matches actual → planner chooses Hash Join
-- Query time: 45 seconds → 200ms (back to normal)
```

Cross-References

- 01_explain_basics.md — Reading plan output
- 02_explain_analyze.md — Actual vs estimated rows
- 06_statistics_and_estimates.md — ANALYZE and statistics target
- 04_join_algorithms.md — Join algorithm selection
- ../07_Indexes/01_index_fundamentals.md — How planner uses indexes